Real-Time Safe and Smooth Visual Servoing for Robot Manipulators via Reinforcement Learning

Jianliang Mao, *Member, IEEE*, Jun Hu, Xin Zhou, *Member, IEEE*, Chuanlin Zhang, *Senior Member, IEEE*, Jun Yang, *Fellow, IEEE*, and Hesheng Wang, *Senior Member, IEEE*

***Abstract*—Real-time collision avoidance is a critical challenge in visual servoing control for safe robot manipulation. This article presents a model-free framework that integrates reinforcement learning (RL) with real-time collision detection (RCD), enabling secure and smooth visual servoing in dynamic environments. The reward function in the RL framework is carefully designed to simultaneously minimize tracking errors, promote motion smoothness, and enforce collision avoidance. Beyond policy optimization through RL, a two-step real-time trajectory blending (RTB) scheme, consisting of a Butterworth filter (BF) and polynomial interpolation, is developed to further refine the generated trajectories and ensure smooth joint velocity profiles. Furthermore, the soft actor-critic (SAC) algorithm is employed for policy learning, and the performance is compared with two widely used methods, namely proximal policy optimization (PPO) and twin delayed deep deterministic policy gradient (TD3). Experimental results on a 6-degree-of-freedom (6-DoF) robot manipulator demonstrate the effectiveness of the proposed approach in achieving safe and smooth visual servoing in real time.**

***Index Terms*—Collision avoidance, motion planning, reinforcement learning (RL), visual servoing.**

Received 3 May 2025; revised 13 September 2025 and 6 November 2025; accepted 9 November 2025. Date of publication 11 December 2025; date of current version 11 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62203292 and Grant 62173221; and in part by the Ministry of Education Chunhui Plan Cooperative Research Projects under Grant 202200897. *(Corresponding author: Xin Zhou.)*

Jianliang Mao and Xin Zhou are with the College of Automation Engineering, Shanghai University of Electric Power, Shanghai 200090, China (e-mail: jl_mao@shiep.edu.cn; xinzhou@shiep.edu.cn).

Jun Hu and Chuanlin Zhang are with the College of Computer Science and Technology, Shanghai University of Electric Power, Shanghai 200090, China (e-mail: Jun_Hu@mail.shiep.edu.cn; clzhang@shiep.edu.cn).

Jun Yang is with the Department of Aeronautical and Automotive Engineering, Loughborough University, LE11 3TU Loughborough, U.K. (e-mail: j.yang3@lboro.ac.uk).

Hesheng Wang is with the School of Automation and Intelligent Sensing, Key Laboratory of System Control and Information Processing of Ministry of Education, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: wanghesheng@sjtu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TIE.2025.3634401, provided by the authors.

Digital Object Identifier 10.1109/TIE.2025.3634401

I. Introduction

In recent years, as a crucial aid to industrial automation, robot manipulators have been increasingly deployed in various scenarios, including industrial assembly [1], logistics sorting [2], and welding operations [3]. In structured environments, offline motion planning typically meets practical requirements. However, with the increasing applications of robots in unstructured situations, dynamic and complex scenes may lead to higher requirements for vision-based control methods [4]. Visual servoing control is an effective approach to enhance the flexibility of robots in adapting to dynamically changing environments. Over the past decade, it has found widespread application in various fields, including inertially stabilized platforms [5], autonomous vehicles [6], [7], and robot manipulators [8], [9]. The core idea behind these applications is to leverage visual information for autonomous robot movement control.

Toward robot manipulation, many efforts have been dedicated for high-performance control of visual servoing applications. For instance, a fuzzy control method is developed in [10] to realize the adaptive parameter adjustment for optimal performance. In [11], the authors devote to propose a sliding mode controller (SMC) to increase the accuracy of visual servoing for systems with the presence of model uncertainties. Moreover, the model predictive control (MPC) strategy is presented in [12], [13] for the constrained motion space. However, the aforementioned methods are inherently dependent on the precise mathematical modeling of the robotic system, classifying them as model-based control approaches. In scenarios where environmental uncertainties are unpredictable or unobservable, these methods may experience significant performance degradation. This limitation highlights the potential advantages of model-free approaches, which do not rely on explicit system models.

As a way of self-optimization, reinforcement learning (RL) is regarded as a promising method for robot manipulation owing to its ability to learn skills autonomously, especially in unstructured environments [14], [15]. By virtue of the intelligent decision-making capabilities of RL, the Cartesian motion planning with vision-based grasping strategy is proposed in [16], [17]. Considering a variety of difficult industrial insertion tasks with visual inputs and different natural reward

1557-9948 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

specifications, an end-to-end approach based on deep RL is studied in [18]. In addition, the techniques by using demonstrations and self-imitation learning to enhance training efficiency, have been introduced into deep RL to improve the control performance [19]. Note that these existing RL methods primarily focus on addressing grasping problems in unstructured environments. However, such non-real-time approaches are inadequate for effectively managing the tracking of dynamic targets in visual servoing tasks. For the visual tracking applications, [20], [21] introduce the convolutional-network based deep learning for control policy training in the absence of obstacles. However, when there are unknown obstacles in manipulation tasks, the integration of safety mechanisms into RL is imperative. Such an approach ensures not only safe tracking but also precise positioning, enabling robust performance in complex and uncertain environments.

In order to guarantee safety manipulations, real-time collision detection (RCD) and avoidance should be realized in a visual servoing task. One promising approach is to mount sensitive components (e.g., electronic skins presented in [22]) on the robot body to achieve sensitive perception. However, the cost is relatively high with the high-performance sensing requirements. Another category establishes a collision observer by collecting information from robot joint encoders and actuators to realize real-time observations of external collision forces, in which the collision detection is based on the occurrence of collisions [23]. For the initial exploration stage of RL, a large number of collisions will inevitably cause certain damage to the robot. The third approach for collision detection is to describe each link of the robot as a convex object and calculate the distance between each pair of objects in the modeling environment [24]. This method has been released as open source in the famous flexible collision library (FCL). However, the standard collision detection method in FCL is computationally time-consuming. To further enhance detection efficiency, [25] introduces a recursive Gilbert–Johnson–Keerthi (RGJK) algorithm that employs a geometric approach to calculate the distance between rigid bodies in the environment and detect potential collisions.

Building upon the aforementioned observations, a model-free RL framework aimed at real-time collision avoidance in visual servoing control tasks is proposed. The key contribution of this work lies in the integration of a general RL approach with RCD and real-time trajectory blending (RTB) to enable safe operation in dynamic environments. Specifically, the proposed design procedure consists of three main steps, as illustrated in Fig. 1. First, a real-world experimental setup is established for motion trajectory collection and validation. Second, a digital twin environment incorporating RCD is developed to facilitate RL training. Third, an RL policy, augmented with RTB by using obstacle locations as part of the observation space, is trained to operate safely within dynamic settings. To optimize the learned policy, a reward function is carefully crafted to simultaneously minimize tracking errors, ensure motion smoothness, and promote collision avoidance. Compared to existing approaches, the primary distinguishing features of the proposed method are summarized as follows.

1) A generic model-free framework is proposed for safe and smooth visual servoing control, enabling simultaneous target tracking and real-time collision avoidance in dynamic environments without relying on prior models of the robot or obstacles.
2) To improve motion smoothness, the proposed approach integrates maximum-entropy RL with a reward-driven RTB mechanism that penalizes abrupt velocity variations and refines policy outputs, thereby ensuring smooth joint velocity profiles for physical manipulators.
3) For efficient policy training and validation, a digital-twin simulation environment integrating the RCD and RTB modules is constructed, effectively bridging the sim-to-real gap and facilitating a seamless transition to real-world robotic systems. Experimental results are demonstrated in the verification video at https://youtu.be/15G-pYg2ohs.

The remainder of the article is organized as follows. Section II presents the preliminaries and problem formulation. Section III details the design procedure of the proposed RL-based visual servoing controller. The experimental results are discussed in Section IV. Finally, Section V provides the conclusion.

II. Preliminaries and Problem Formulation

In this section, a brief review of maximum entropy RL is provided, which serves as one of the key techniques in the proposed framework. Subsequently, the safe visual servoing control problem is formulated within the RL setting.

A. Preliminaries

1) Maximum Entropy RL: Different from the standard RL where the policy is to maximize a long-term reward, maximum entropy RL aims to maximize the expected return and the policy entropy at the same time [26], where the optimal policy π^* is defined as

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \sum_{t=0}^{T} \gamma^t \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r(s_t,a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))] \quad (1)$$

where γ^t represents the discount factor, s_t and a_t are the continuous state and action at time-step t, respectively, ρ_π is the state–action distribution induced by the policy π, T is the maximum time steps, $r(s_t,a_t)$ is the immediate reward, $\mathcal{H}(\cdot)$ is the entropy term, and α is a temperature parameter used for entropy adjustment.

Following the policy π, the state–action value function Q^π_{soft} and the state–value function V^π_{soft} are defined as

$$Q^\pi_{\text{soft}}(s,a) = \mathop{\mathbb{E}}_{s_t,a_t\sim\rho_\pi}\Big[\sum_{t=0}^{\infty}\gamma^t r(s_t,a_t) \quad (1)$$

$$+ \alpha\sum_{t=1}^{\infty}\gamma^t\mathcal{H}(\pi(\cdot\mid s_t)) \mid s_0 = s, a_0 = a\Big] \quad (2)$$

$$V^\pi_{\text{soft}}(s) = \mathop{\mathbb{E}}_{s_t,a_t\sim\rho_\pi}\Big[\sum_{t=0}^{\infty}\gamma^t\Big(r(s_t,a_t)$$

$$+ \alpha\mathcal{H}(\pi(\cdot\mid s_t))\Big) \mid s_0 = s\Big] \quad (3)$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

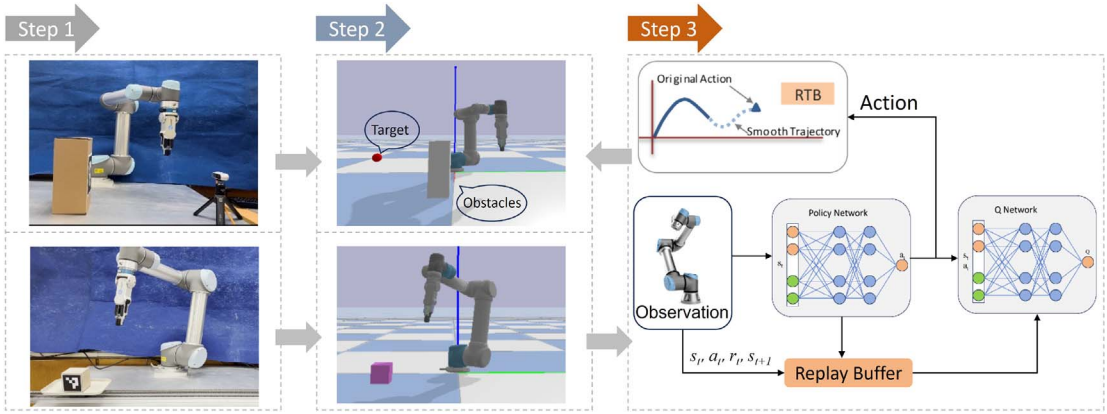


Fig. 1. An illustration of the proposed design procedure. In step 1, a real-world robot manipulator is constructed for motion trajectories collection and experimental verification. In step 2, building a digital twin environment for RL training with RCD. In step 3, training the RL policy with RTB by utilizing the location of obstacles as an observed state.

where s_0 and a_0 represent the initial state and initial action, respectively.

To capture multiple modes of near-optimal policies, the maximum entropy RL uses a unique energy-based policy (EBP)

$$\pi\left(a_t \mid s_t\right) \propto \exp\left(-\mathcal{E}\left(s_t, a_t\right)\right) \tag{4}$$

where $\mathcal{E}(\cdot)$ is an energy function [27]. In practice, since the tractable policies are preferred in the training process, the Gaussian distribution is usually developed to interact with the environment. Subsequently, it is common to use the Kullback–Leibler (KL) divergence to narrow the distance between EBP and the Gaussian distribution with an optimal policy.

B. Problem Formulation

For visual servoing control of robot manipulators, safe target tracking or positioning is an indispensable task. In this article, we aim to realize real-time safe visual servoing control of robot manipulators in 3-D space. Concretely, given the current observation s_t (e.g., the robot and environment states) at time t, the objective is to determine a proper joint-space command a_t such that the current end-effector pose p_t accurately tracks the desired pose g_t. Note that after executing the action command a_t, the robot will transition to a new state s_{t+1} with the probability $p(s_{t+1}|s_t, a_t)$. This control problem in essence amounts to a Markov decision process (MDP) problem, which can be naturally solved using RL. For the observation s_t, the RL policy samples an action a_t from a stochastic policy π: $a_t \sim \pi_\theta(\cdot|s_t)$, where the optimal policy is obtained by maximizing

$$\pi_\theta^* = \underset{\pi_\theta}{\operatorname{argmax}} \sum_{t=0}^{T} \gamma^t \mathbb{E}\left[r_t + \alpha \mathcal{H}(\pi_\theta)\right] \tag{5}$$

where r_t is a composite indicator at time-step t to be designed.

To effectively implement this model-free control framework in real-time, it is essential to incorporate strategies that ensure safety and achieve smooth motion in dynamic environments, which serves as the motivation for this work.

III. RL-Based Visual Servoing Control Design

In this section, to realize safe visual servoing control of robot manipulators, a model-free RL framework is introduced that integrates RCD and trajectory blending, enabling the tracking of dynamic targets. An overview of the proposed framework is shown in Fig. 2, and the detailed design procedure is presented as follows.

A. Real-Time Collision Detection

To realize flexible collision avoidance, including self-collisions and collisions with obstacles, the RTB is implemented by the following two steps.

1) Environment Modeling: To describe the spatial location of robot components conveniently, a graphical data structure based on a spatial grid is constructed. Each component in the robot workspace is assigned a coordinate system to represent its spatial pose, and link relationships are defined through transformation matrices relative to a given reference frame. Combined with the forward kinematics of the robot, any pose can thus be calculated and modeled. The unified robot description format (URDF) is adopted to model both the manipulator and its installation environment, as shown in Fig. 1, where each component is represented using the standard triangle language (STL). This modeling approach is robot-agnostic, requiring only the manipulator's kinematic model from CAD or URDF files.

In terms of obstacle handling, randomized obstacles with full state information are generated during training, which is a common practice in RL to facilitate efficient policy learning. The randomization includes variations in position and size, thereby enhancing the generalization capability of the learned policy. In real-world experiments, obstacle states must be obtained through perception using an Intel RealSense camera, where obstacles are approximated by rectangular bounding boxes to simplify detection and ensure reliable localization. For complex obstacles, voxel-based reconstruction [28] can be combined with convex decomposition, and the resulting convex parts are then processed by the RCD module.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

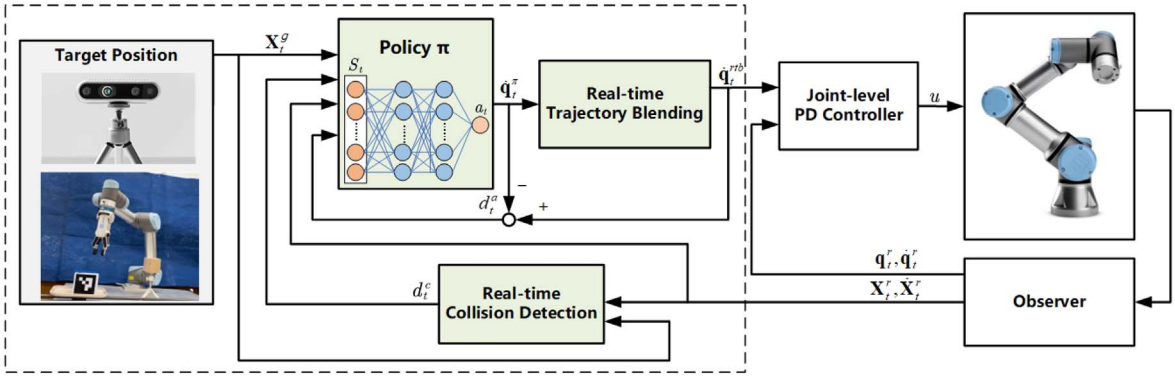


Fig. 2. An overview of the proposed RL-based visual servoing control architecture. $\mathbf{X}_t^g$ represent the target pose, $\mathbf{X}_t^r$ and $\dot{\mathbf{X}}_t^r$ denote the end-effector pose and velocity, respectively, $\mathbf{q}_t^r$ and $\dot{\mathbf{q}}_t^r$ denote the joint position and velocity, respectively, $\dot{\mathbf{q}}_t^\pi$ is the policy output and $\dot{\mathbf{q}}_t^{rtb}$ is the reference joint velocity generated by the RTB process, u denotes the control signal, d_t^a is the prefetched safe distance generated by the RTB module, and d_t^c is the actual safe distance.

2) Distance Calculation: In what follows, the closest distance between one component and the others is calculated by iteration. After the modeling in the previous steps, each link of the robot and obstacles can be described as the convex object. For any two objects, the point sets $\mathbf{P}_1$ and $\mathbf{P}_2$ can be defined to represent them. Define the Minkowski difference between $\mathbf{P}_1$ and $\mathbf{P}_2$

$$\mathbf{P}_1 \ominus \mathbf{P}_2 = \{\mathbf{p}_1 - \mathbf{p}_2 : \forall \mathbf{p}_1 \in \mathbf{P}_1, \forall \mathbf{p}_2 \in \mathbf{P}_2\}. \tag{6}$$

Note that if the result of (6) contains the origin $\mathbf{O}$ of coordinates, it means that the two objects have overlapped, and a collision has occurred. Otherwise, the shortest distance between $\mathbf{P}_1$ and $\mathbf{P}_2$ is denoted by

$$d_t^c = \min\{\|\mathbf{c} - \mathbf{O}\| : \forall \mathbf{c} \in \{\mathbf{P}_1 \ominus \mathbf{P}_2\}\} \tag{7}$$

where $\mathbf{c}$ represents a point in the Minkowski difference set.

To calculate d_t^c, the RGJK algorithm developed in [25] is employed, as shown in Algorithm 1. The initial value of d_t^c is set as a threshold that depends on the specific application. The RGJK algorithm extends the Gilbert–Johnson–Keerthi (GJK) method by using a geometric approach to compute the distance between rigid bodies in the environment and detect potential collisions. Compared to the traditional FCL that is computationally time-consuming, the RGJK algorithm is more efficient and generic. This is due to it requires only the robot's URDF and the object meshes with their respective transformations. Further details can be found in [25].

Remark 1: It should be noted that RCD requires obstacles to remain observable. When the field of view is limited or temporary occlusion occurs, robustness can be improved through short-horizon trajectory prediction for slowly moving obstacles. Although not addressed in this work, future extensions may incorporate temporal models such as long short-term memory (LSTM) or transformer-based predictors [29] and uncertainty-aware mapping methods [30], where deep neural networks estimate occupancy beyond the current sensor range and exploit uncertainty to maintain conservative safety under partial observability.

Algorithm 1: Real-Time RGJK Collision Detection.

Input: Robot joint angles $\theta_1, \theta_2, \ldots, \theta_6$, and the position of obstacles $P_7, \ldots, P_N$
Output: Shortest distance d_t^c
$\mathbf{P}_1, \mathbf{P}_2, \ldots, \mathbf{P}_N$ are updated positions
for $i = 1$ to N **do**
  for $j = i + 1$ to N **do**
    Compute the distance $d_{ij} = \mathrm{RGJK}(\mathbf{P}_i, \mathbf{P}_j)$
    if $d_{ij} \leq d_t^c$ **then**
      $d_t^c = d_{ij}$
    end if
  end for
end for

B. Reinforcement Learning

In this section, the soft actor-critic (SAC) method [26] based on the maximum entropy RL is used to search for the optimal policy. The choice of SAC over other RL algorithms is motivated by the requirements of real-time visual servoing, where efficient policy learning, stable adaptation to dynamic targets, and safety-aware control are critical. Compared with proximal policy optimization (PPO), SAC offers substantially higher sample efficiency and more reliable long-term performance, which is particularly important for real-time adaptation under constrained training resources. In contrast to the twin delayed deep deterministic policy gradient (TD3), SAC benefits from entropy regularization and automatic temperature adjustment, providing more stable value estimation and greater robustness to hyperparameter variations. Together, these advantages make SAC well suited for safety-aware real-time visual servoing tasks. Design details, including the learning model, state space, action space, and reward function, are introduced step by step.

1) Learning Model: Generally, SAC learns an actor policy π_θ and a critic Q_w. To update the actor, a Gaussian policy distribution is adopted to interact with the environment. The policy parameters are optimized by minimizing the objective

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

function as

$$J_\pi(\theta) = \mathop{\mathbb{E}}_{s_t\sim D, a_t\sim\pi_\theta}\Big[\alpha \log \pi_\theta(a_t \mid s_t) - Q_w(s_t, a_t)\Big] \quad (8)$$

where J_π denotes the policy objective that balances the maximization of the expected return with an entropy-regularization term to encourage exploration, and D is the replay buffer.

To reduce the variance of gradient estimation, the reparameterization trick is applied to express the stochastic action as

$$a_\theta(s_t, \epsilon) = \tanh\big(\mu_\theta(s_t) + \sigma_\theta(s_t) \odot \epsilon\big), \quad \epsilon \sim \mathcal{N}(0, I) \quad (9)$$

where $\mu_\theta(s)$ and $\sigma_\theta(s)$ are outputs of the policy network, and $\odot$ denotes the element-wise product. The policy distribution is modeled as a Gaussian with diagonal covariance $\sigma_\theta^2(s)I$, which is not full-rank but is a standard and effective design choice in SAC. This ensures sufficient stochasticity for exploration while preserving numerical stability and training efficiency. It is also important to note that the log-probability $\log \pi_\theta(a|s)$ incorporates the Jacobian correction term introduced by the $\tanh$ transformation. Accordingly, the reparameterized objective becomes

$$J_\pi(\theta) = \mathop{\mathbb{E}}_{s_t\sim D, \epsilon\sim\mathcal{N}}\Big[\alpha \log \pi_\theta(a_\theta(s_t, \epsilon)|s_t) - Q_w(s_t, a_\theta(s_t, \epsilon))\Big]. \quad (10)$$

While optimizing the policy, SAC also trains the critic Q_w by minimizing the soft Bellman residual as

$$J_Q(w) = \mathop{\mathbb{E}}_{(s_t, a_t, s_{t+1})\sim D}\Bigg[\frac{1}{2}\Big(Q_w(s_t, a_t) - \big(r(s_t, a_t) + \gamma\big(Q_{\bar{\omega}}(s_{t+1}, a_{t+1}) - \alpha \log\big(\pi_\theta(a_{t+1} \mid s_{t+1})\big)\big)\big)\Big)^2\Bigg] \quad (11)$$

where $Q_{\bar{w}}$ denotes the target soft Q-function with delayed parameters.

Finally, the temperature parameter α is tuned to match a target entropy $\bar{\mathcal{H}}$ by minimizing

$$J(\alpha) = \mathop{\mathbb{E}}_{s_t\sim D, a_t\sim\pi_\theta}\Big[-\alpha\big(\log \pi_\theta(a_t|s_t) + \bar{\mathcal{H}}\big)\Big]. \quad (12)$$

This update increases α whenever the policy entropy is below the target, ensuring a proper balance between exploration and exploitation throughout training.

2) State Space: The state space consists of the states of the robot manipulator, tracked target and obstacles, of which the full observation is defined as $s_t = [\mathbf{o}_t^r, \mathbf{o}_t^g, \mathbf{o}_t^c]$. The first component $\mathbf{o}_t^r = [\mathbf{q}_t^{r\top}, \dot{\mathbf{q}}_t^{r\top}]$ is a concatenation of the joint position $\mathbf{q}_t^r$ and the joint velocity $\dot{\mathbf{q}}_t^r$. The second component $\mathbf{o}_t^g = [d_t^g, \dot{d}_t^g]$ where $d_t^g = \|\mathbf{X}_t^g - \mathbf{X}_t^r\|^2$ is the difference between the target pose $\mathbf{X}_t^g$ and the end-effector pose $\mathbf{X}_t^r$, $\dot{d}_t^g = \|\dot{\mathbf{X}}_t^g - \dot{\mathbf{X}}_t^r\|^2$ is the difference between the target velocity $\dot{\mathbf{X}}_t^g$ and the end-effector velocity $\dot{\mathbf{X}}_t^r$. The third component $\mathbf{o}_t^c = [d_t^a, d_t^c]$ where $d_t^a = \|\dot{\mathbf{q}}_{t-1}^{rtb} - \dot{\mathbf{q}}_{t-1}^{\pi}\|^2$ is the difference between the policy output $\dot{\mathbf{q}}_{t-1}^{\pi}$ and the reference joint velocity $\dot{\mathbf{q}}_{t-1}^{rtb}$ from the last time step. The observations are normalized by subtracting the mean and dividing by the standard deviation using the statistics aggregated over the course of the entire training.

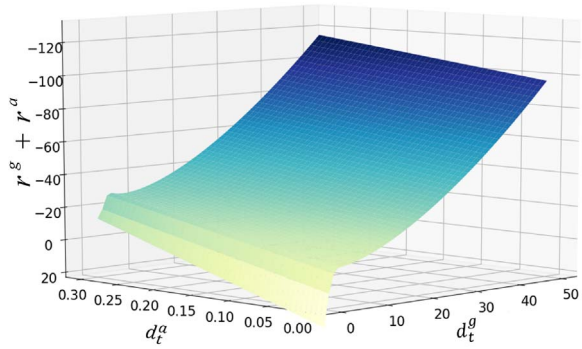


Fig. 3. Variations of the reward function as the robot approaches the target.

3) Action Space: The action space a_t consists of a set of joint velocities, expressed by $\dot{\mathbf{q}}_t^{\pi}$. Note that singularity problems can be avoided by selecting actions in joint space as the control inputs. For more reliable training, the range of the action value is set within [–1, 1] rad/s, and the maximum change of each control cycle is set to 0.1 rad/s. On the other hand, in order to obtain a smoother motion, a real-time velocity blending (VB) controller is designed, which will be introduced in the following section.

4) Reward Design: The reward function r_t that considers tracking errors, motion smoothness as well as collision avoidance is defined as

$$r_t = r_t^g + r_t^a + r_t^c. \quad (13)$$

In (13), the term r_t^g depends on the tracking error

$$r_t^g = \begin{cases} c_1^g (d_t^g - \mathcal{C}_g)^{c_2^g}, & \text{if } d_t^g \le \mathcal{C}_g \\ -c_3^g (d_t^g - \mathcal{C}_g)^{c_4^g}, & \text{if } d_t^g > \mathcal{C}_g \end{cases} \quad (14)$$

where $\mathcal{C}_g$ is the threshold, and $c_i^g, i = 1, \ldots, 4$ are positive weight coefficients.

The term r_t^a measures the smoothness of actions

$$r_t^a = -\frac{c_1^a d_t^a}{\big(\max(d_t^g, 1)\big)^{c_2^a}} \quad (15)$$

where $c_j^a, j = 1, 2$ are positive weight coefficients.

The term r_t^c is a penalty for the occurrence of collisions

$$r_t^c = \begin{cases} -r_{\text{collision}}, & \text{if } d_t^c \le \mathcal{C}_c \\ 0, & \text{otherwise} \end{cases} \quad (16)$$

which means that if the safety distance d_t^c is less than the threshold $\mathcal{C}_c$, the policy will get a large penalty $r_{\text{collision}}$, such that r_t^c can almost mask the other two reward terms in the presence of collisions.

Fig. 3 illustrates how the reward function evolves as the robot approaches the target. Notably, by reasonably adjusting the parameters c_i^g and c_j^a, the robot achieves enhanced exploratory ability when the end-effector is distant from the target, and improved tracking performance with greater robustness when the end-effector is close to the target.

It is worth noting that the hyperparameters in the reward function are typically determined through extensive training

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: SAC-Based RL Training.

Input: environmental state s_t
Output: action a_t
Initialize online Q network, target Q network, and policy network with weights ω, $\bar{\omega}$, θ;
Build target motion trajectory dataset $\mathcal{D}$;
Initialize replay buffer $\mathcal{R}$;
Check collision information in real-time;
for each iteration **do**
  Sample a random target trajectory D_i form dataset $\mathcal{D}$;
  Initialize robot to a collision-free pose randomly;
  Sample initial state $s_0 = [o_0^r, o_0^g, o_0^c]$, which is related to D_i;
  for each environment step **do**
    $a_t = \text{Policy}(s_t)$;
    $s_{t+1} = \text{Env.step}(a_t, s_t)$;
    $r_t = \text{Reward}(s_t, a_t)$;
    $\text{Add}(s_t, a_t, r_t, s_{t+1})$ into $\mathcal{R}$;
    if collision or success **then**
      break;
    end if
  end for
end for
for each gradient step **do**
  Update online Q network weights: $\omega \leftarrow \omega - \lambda_Q \hat{\nabla}_\omega J_Q(w_i)$;
  Update policy weights: $\theta \leftarrow \theta - \lambda_\pi \hat{\nabla}_\theta J_\pi(\theta)$;
  Adjust temperature: $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha J(\alpha)$;
  Update target Q network weights: $\bar{\omega} \leftarrow \tau\omega + (1-\tau)\bar{\omega}$;
end for

and systematic trial-and-error to achieve optimal values. The detailed tuning process is presented in the experimental section. The detailed realization of RL training is summarized in Algorithm 2.

C. Real-Time Trajectory Blending

To avoid the jitter behavior of action values generated by RL, a RTB strategy is proposed to modify the direct output of the policy. Specifically, the RTB is made up of two aspects.

1) Low-Pass Butterworth Filter: A low-pass butterworth filter (BF) is firstly adopted to smooth the action value and improve the robustness of the RL policy. The detailed expression of the filter is

$$\dot{\mathbf{q}}_t^{bw} = \frac{\dot{\mathbf{q}}_t^{\pi} + \dot{\mathbf{q}}_{t-1}^{\pi} - \left(1 - 2/(\omega_c T_s)\right)\dot{\mathbf{q}}_{t-1}^{bw}}{1 + 2/(\omega_c T_s)} \tag{17}$$

where ω_c is the cut-off frequency, T_s is the sampling period, $\dot{\mathbf{q}}_t^{\pi}$ is the policy output, and $\dot{\mathbf{q}}_t^{bw}$ is the action after the low-pass filter.

2) Velocity Blending: Second, a VB scheme is developed to generate smooth joint velocity trajectories. Suppose that at the time step t_1, the joint velocity is denoted as $\dot{\mathbf{q}}_{t_1}^{r}$ and the RL policy gives the action denoted as $\dot{\mathbf{q}}_{t_1}^{bw}$. Within the next time step t_2, the transition of the joint velocity $\dot{\mathbf{q}}_{\iota}^{rtb}$ can be computed by

$$\dot{\mathbf{q}}_{\iota}^{rtb} = \dot{\mathbf{q}}_{t_1}^{r} + \alpha(s)(\dot{\mathbf{q}}_{t_1}^{bw} - \dot{\mathbf{q}}_{t_1}^{r}), \iota \in [t_1, t_2] \tag{18}$$

where $\alpha(s)$ is the blending function and $s = (\iota - t_1)/(t_2 - t_1)$ is the normalized time parameter. As a boundary condition, $\alpha(s)$ is defined as $\alpha(s) = 6s^5 - 15s^4 + 10s^3$ in our designed approach. Therefore, the velocity trajectory direction is corrected in real time by considering the latest way points of the RL model at each time-step, as shown in Fig. 4.

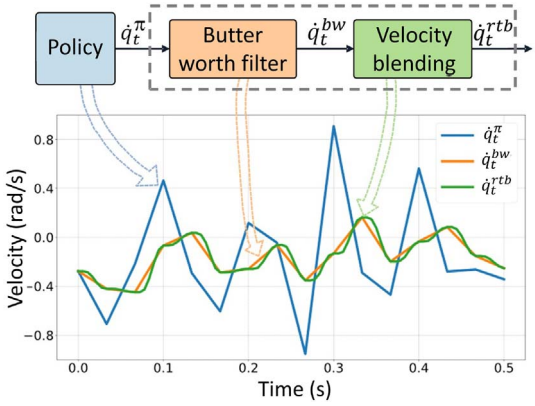


Fig. 4. An illustration of the RTB scheme.

Remark 2: Although many trajectory smoothing methods have been proposed, the aim of this work is not to introduce a new technique but to integrate trajectory blending into an RL-based framework for real-time visual servoing, with smoothness and real-time performance as the main considerations. A BF is adopted to suppress jitters in the RL-generated joint velocities, ensuring the feasibility of subsequent velocity interpolation, while other filters can be selected in practice depending on the environment. In addition, the RL reward function (13) includes a smoothness term (15) that penalizes abrupt velocity variations, enabling the agent to learn smoother control strategies. By combining reward-driven smoothness with execution-level refinement through a lightweight two-step scheme of Butterworth filtering and polynomial interpolation, the proposed RTB achieves both computational efficiency and reliable trajectory profiles suitable for real-robot deployment.

Remark 3: It should be highlighted that this work focuses on real-time visual servoing in environments with randomly placed static obstacles, as reflected in the experimental design. Under these conditions, the RGJK-based RCD, combined with the learned policy and the RTB, ensures safe and smooth operation. However, for dynamic obstacles, more reliable avoidance requires the integration of predictive capabilities in addition to perception. As part of our ongoing research, we are investigating short-horizon trajectory prediction with temporal models (e.g., LSTM) and proactive safety enforcement using control barrier functions (CBFs) to enhance robustness in rapidly changing environments. Preliminary results in this direction can be referred to in our recent work [31].

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

IV. Experimental Evaluation

In this section, a series of experiments in both simulated and real robotic systems is carried out to demonstrate the efficacy of the proposed approach.[1]

A. Environmental Setup

For the training environment, all simulations are conducted in the PyBullet simulator, which provides a safe and high-fidelity environment that closely replicates the physical system to train the optimal policy. The Bullet physics engine is applied to the simulations, where a 6-degree-of-freedom (6-DoF) Universal Robot 5 (UR5) is modeled for visual servoing tasks. A Python interface is employed between the RL algorithm and the agent environment. The policy is trained on a computer with an Intel i7-10750H CPU and an Nvidia RTX 4050 GPU.

For the real-world validation, the UR5 manipulator is operated in velocity-control mode through the official UR controller interface. Two Intel RealSense D435i RGB-D cameras are employed, with one fixed overlooking the robot workspace for obstacle detection and another designated for dynamic target recognition and tracking. The vision system operates at an average update rate of 60 Hz. The control loop is implemented externally on an industrial PC with an Nvidia RTX 4050 GPU, where the control policy is computed and transmitted to the robot in real time via Ethernet. To evaluate the effectiveness of the trained policy, two representative tasks are performed: a reaching task and a dynamic target tracking task. In the reaching task, the target position is defined as a desired pose, whereas in the dynamic tracking task, it is obtained through visual detection of the designated object. Obstacles are automatically detected from the environment using the visual system, with their geometric information extracted through image processing and used by the RCD module to compute minimum distances for collision avoidance.

B. Training Setup

1) *Obtaining State Information:* To begin with, during the training process, the position information of the target points and obstacles, as well as the joint information of the robot manipulator, is obtained through the application programming interface (API) provided by the Pybullet simulator. In the real environment, position information is acquired through an RGB-D camera, while joint information of the robotic system is obtained in real-time via the API provided by the UR5 robot. Subsequently, the obtained position information of the target points and joint information from the simulation state are utilized as the state space for training.
2) *Parameter Selection:* For the network architecture, both the actor and critic networks are implemented as multilayer perceptrons (MLPs) with four layers. The input layer processes a state vector of dimension 30. Two hidden layers follow, with dimensions of 256 and 64, respectively. The output layer differs between the networks: the policy network outputs an action vector of dimension 6, while the Q-network outputs a scalar representing the estimated state-action value.
   During training, the influence of hyperparameter settings on policy performance is systematically examined. In the early stages, reasonable parameter ranges are identified through preliminary experiments and analysis of reward convergence trends. These insights are then used to iteratively refine the hyperparameters to achieve stable convergence and improved performance. Although some trial-and-error adjustments are required initially, the process becomes increasingly systematic as the impact of key parameters is better understood, thereby streamlining fine-tuning and reducing redundancy. The final parameter settings are summarized in Table I.
3) *Training Process:* As outlined in Algorithm 2, the training process begins with a series of initialization steps, including the setup of RCD. Convergence is evaluated by monitoring the stabilization of episodic rewards across training episodes. Key hyperparameters, such as the learning rate, batch size, discount factor, and exploration parameters. The control framework operates at multiple frequencies: the control policy runs at 30 Hz, while the proposed RTB module functions as an intermediate layer at 100 Hz to provide interpolated data to the PD controller. Each episode comprises a maximum of 3000-time steps, with the training process spanning 5000 episodes. During training, a replay buffer collects interaction data from the robot's environment, facilitating simultaneous updates to the neural networks. The entire training process is completed in approximately 8 h.
4) *Sim-to-Real Transfer:* During the transition of the model from a simulated environment to the real world, a series of system configurations is required. The depth camera captures data at a frame rate of 60 Hz and transmits the information in real time via the USB protocol. The UR5 operates at a control frequency of 100 Hz. The host machine synchronously collects data from both the camera and the robotic arm at 30 Hz, feeds the information into the model for inference, and outputs the corresponding control policy at the same frequency. Since the APIs invoked by the host and the neural network inference operate at significantly higher speeds than the data acquisition and control frequencies, and given that processing performance can be further improved with more powerful computational hardware, the overall system latency can be considered negligible.

C. Safe Reaching Task

1) *Task Description:* The reaching task is a fundamental component in robot manipulation that requires the end-effector to arrive at a target pose. In this case, we train the UR5 for set-point tracking with obstacle avoidance. Collision avoidance includes self-collisions and collisions with obstacles in the environment. The UR5 follows the

[1]Please refer to the supplementary material for a video demonstration illustrating the evaluation details.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS OF SAC TRAINING

Parameter	Definition	Value
Θ	Policy network size	$30 \times 256 \times 64 \times 6$
Ω	Q Network size	$30 \times 256 \times 64 \times 1$
γ	Discount factor	0.95
M	Minibatch size	256
λ	Learning rate	$3e^{-4}$
$\mathcal{R}$	Size of replay buffer	$10e^5$
$\mathcal{C}_g$	Threshold of the r^g	3
$\mathcal{C}_c$	Threshold of the r^c	2
c_1^g, c_2^g, c_3^g, c_4^g	Weight of the r^g	4, 2, 0.3, 1.5
c_1^a, c_2^a	Weight of the r^a	110, 0.15
$r_{\text{collision}}$	Penalty for collision	400
ω_c	Cut-off frequency	30

policy in taking actions to reach the goal pose, and the positions of the goal and obstacles are sampled from the dataset. The obstacle side length dimension is set to the range [0.1, 0.3] m and the spatial location of the obstacle is constrained from $[0.1, 0.1, 0.15]^\top$ m to $[0.1, 0.2, 0.15]^\top$ m. The testing procedure can be divided into the following steps. Firstly, the position of the obstacle is randomly initialized within the constraint space. Secondly, the robot manipulator is initialized with respect to the left side of the obstacle. Thirdly, a collision-free point is sampled from the right side of the obstacle as the target position.

2) *Training Results:* To further evaluate the effectiveness of the proposed method, PPO and TD3 algorithms are employed for comparative analysis. We implement four different strategies by combining the BF and VB modules. As shown in Fig. 5, the efficiency of the algorithms does not decrease regardless of whether BF or VB is added, and across all four combinations, the reward functions converge to nearly the same level. It is worth noting that PPO often exhibits faster initial convergence due to its on-policy updates, whereas SAC, benefiting from its off-policy design with a replay buffer, achieves higher overall sample efficiency and more reliable long-term performance. This distinction highlights the suitability of SAC for real-time visual servoing tasks where both efficiency and stability are essential. Furthermore, Table II illustrates the success rates (SRs) of various algorithms under different combinations over 1000 rounds of testing. As depicted, when the algorithms are used individually, the SRs of training are relatively low. However, upon incorporating the BF and VB modules, the SRs of all three algorithms show improvement.

3) *Real-World Tracking Results:* As shown in Fig. 6, all three methods can successfully avoid the obstacle and reach the target pose. In comparison, the proposed SAC-based control framework demonstrates the optimal path length when crossing the obstacle. Meanwhile, it can be observed from Fig. 7 shows the real-time tracking errors of the end-effector under the safe reaching task using three different strategies. The results are reported as mean values with shaded areas representing the standard deviation of 30 repeated trials, which more clearly illustrate the variability of the results. It can be observed that the tracking error decreases consistently over time for all methods. Moreover, the proposed SAC-based strategy achieves faster convergence and reaches the target point in the shortest time, demonstrating its superior effectiveness compared to PPO and TD3.

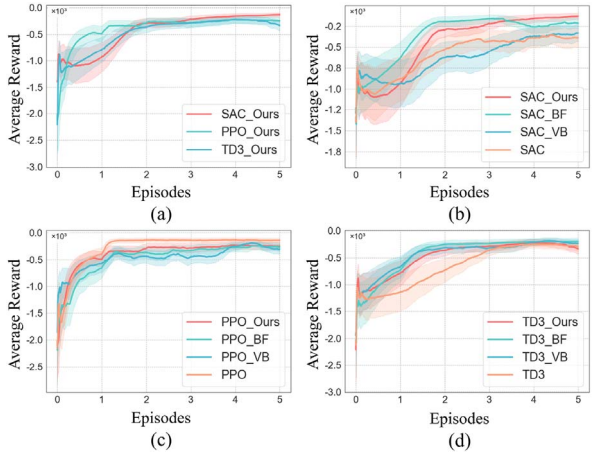


Fig. 5. Average rewards during the training process with different strategies: (a) SAC_Ours, PPO_Ours, and TD3_Ours with the combination of BF and VB; (b) different SAC strategies with or without BF and VB; (c) different PPO strategies with or without BF and VB; and (d) different TD3 strategies with or without BF and VB.

TABLE II
COMPARISON OF SRS UNDER SAFE REACHING TASK

Policy	SR (%)	Policy	SR (%)	Policy	SR (%)
SAC_Ours	**77.29**	**PPO_Ours**	**81.79**	**TD3_Ours**	**75.63**
SAC_BF	75.04	PPO_BF	71.45	TD3_BF	75.57
SAC_VB	76.75	PPO_VB	75.23	TD3_VB	74.62
SAC	75.36	PPO	71.43	TD3	48.32

Note: Bold entries represent the best indicator.

Furthermore, safe reaching experiments with multiple obstacles are conducted to evaluate the generalization capability of the framework. As shown in Fig. 8, both SAC and PPO successfully achieve collision-free reach, with SAC generating smoother trajectories, while TD3 exhibits sensitivity to obstacle configurations. After changing the obstacle settings, TD3 can also complete the task, indicating a relatively weak generalization performance. The results in Fig. 9 are consistent with those in Fig. 7, further confirming that the proposed method remains effective in multiobstacle scenarios.

To further evaluate motion smoothness, Fig. 10 compares the joint velocity profiles of different strategies. The velocities of wrist joints 1, 2, and 3 remain close to zero during motion since the end-effector pose scarcely changes. As illustrated, the application of the BF module results in noticeably smoother velocity profiles. With the additional incorporation of the VB module, smoothness is further improved and jitter is almost eliminated. The same framework can also be applied to traditional PPO and TD3 algorithms.

Finally, the computational overhead introduced by the RTB module is evaluated. On our experimental platform, the control policy runs at 30 Hz with an average inference time of about 5 ms per cycle, while the RTB module adds only about 0.1 ms

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

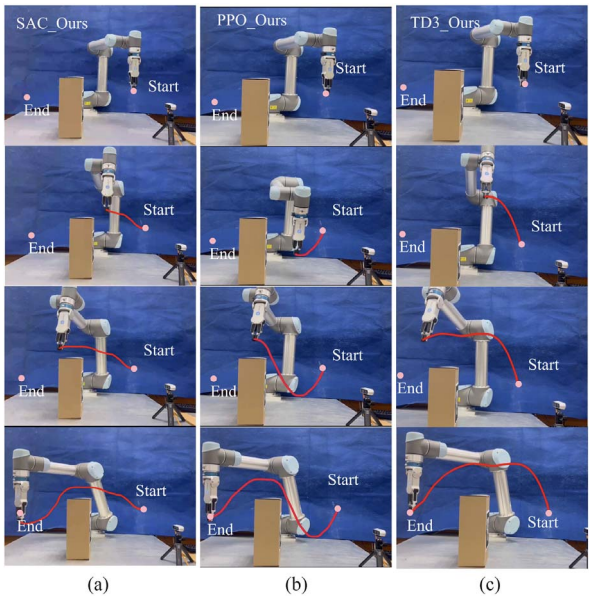


Fig. 6. Frame-by-frame plots under the safe reaching task with one obstacle using three different strategies: (a) SAC_Ours with the combination of BF and VB; (b) PPO_Ours with the combination of BF and VB; and (c) TD3_Ours with the combination of BF and VB.

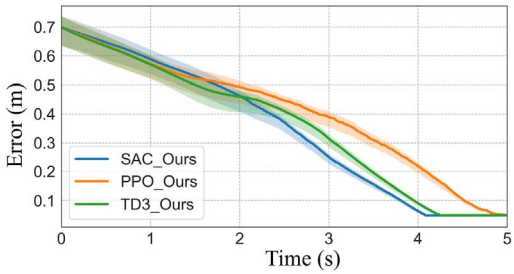


Fig. 7. Real-time tracking errors of the end-effector under the safe reaching task with one obstacle using three different strategies.

per cycle. This additional cost is negligible within the control loop and does not impose any extra computational burden. Moreover, the average task completion time is assessed across five randomly obstacle scenarios, as shown in Fig. 11. The proposed approach achieves an average of 5.0 s compared to 4.2 s with plain SAC, where the slight increase is attributed to the smoother motions generated by the RTB scheme. Importantly, these results confirm that the proposed framework maintains both efficiency and reliability.

D. Safe Dynamic Target Tracking Task

1) *Task Description:* We further extend the scenario to dynamic target tracking. The workspace is depicted in Fig. 12. The tracked target is abstracted as a cube. Additionally, another collision scenario is introduced to demonstrate the effectiveness of the proposed method in avoiding collisions with dynamic targets. The initial pose of the UR5 is set to a randomly selected collision-free position. Different motion trajectories and target sizes are applied in each round via sampling from the dataset. The objective is to use RL to train a velocity control action, and then control the UR5 in a continuous state-action space to achieve precise tracking of dynamic targets while avoiding collisions.

2) *Training Results:* Different from the reaching task, dynamic target tracking is more complex because it involves continuously adjusting the position and velocity of the target over time. While SAC may be less computationally efficient due to entropy regularization, it consistently outperforms PPO and TD3, which face challenges such

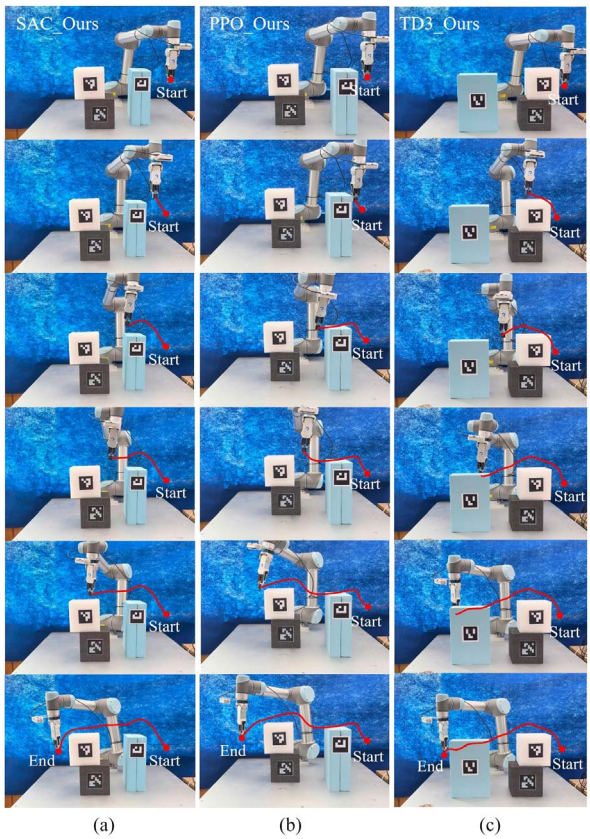


Fig. 8. Frame-by-frame plots under the safe reaching task with multiple obstacles using three different strategies: (a) SAC_Ours with the combination of BF and VB; (b) PPO_Ours with the combination of BF and VB; and (c) TD3_Ours with the combination of BF and VB.

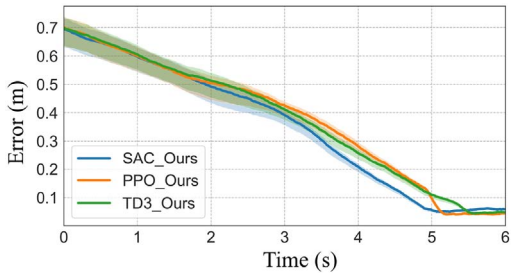


Fig. 9. Real-time tracking errors of the end-effector under the safe reaching task with multiple obstacles using three different strategies.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

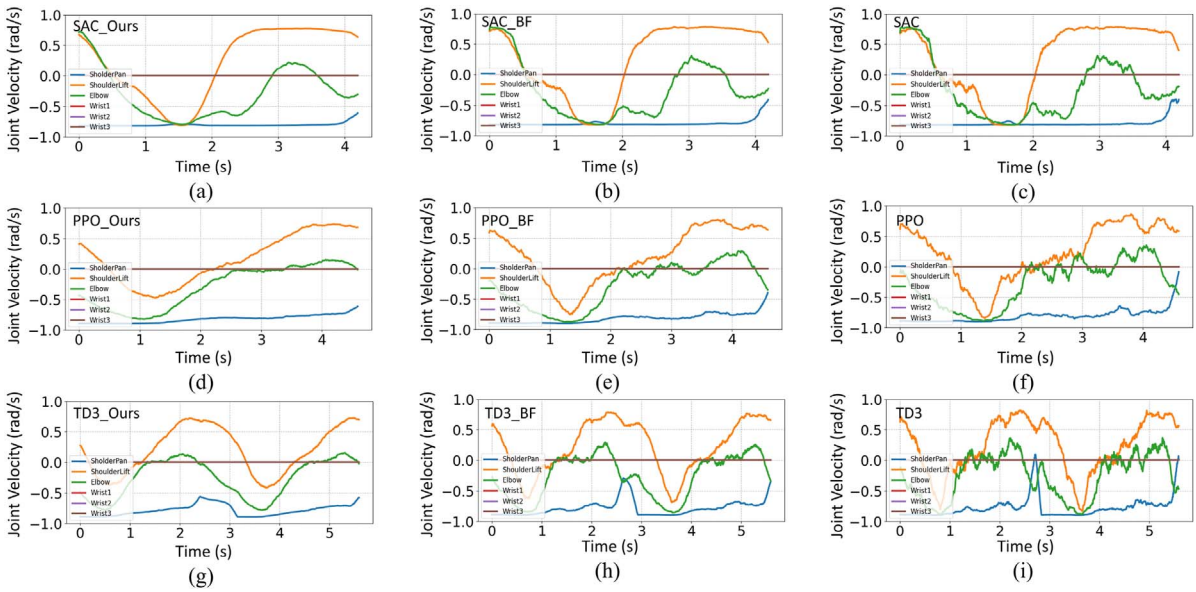


Fig. 10. Comparative analysis of robot joint velocity profiles with different strategies: (a)–(c) depict the joint velocity profiles using three different combinations of SAC; (d)–(f) depict the joint velocity profiles using three different combinations of PPO; and (g)–(i) depict the joint velocity profiles using three different combinations of TD3.

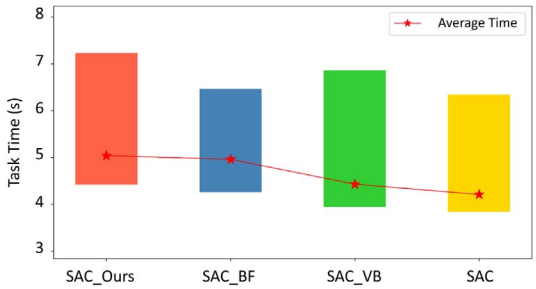


Fig. 11. Average task completion time for four different combinations.

as local optima, poor generalization, and hyperparameter sensitivity in dynamic environments [32]. Indeed, the entropy maximization in SAC helps balance exploration and exploitation, preventing premature convergence and improving generalization. As shown in Table III, our SAC approach achieves the highest SR over 1000 rounds of testing, making it particularly effective for dynamic complex tasks like target tracking in visual servoing.

3) *Real-World Tracking Results:* Fig. 12 illustrates the tracking process of the robot manipulator during the forward and reverse movement of a dynamic target. At the initial moment, the end-effector is more than 15 cm away from the target, which moves continuously at a speed of 0.1 m/s. The results demonstrate that the robot accomplishes the tracking task using the proposed SAC-based approach. Fig. 13 shows the tracking error during the process. The error initially decreases, then rises, and subsequently decreases again. This behavior is mainly due to the obstacle placed along the tracking path, which forces the end-effector to increase the error to avoid collision.

In addition to the 1-D tracking experiments, 2-D dynamic target tracking tasks with variable speed are also performed. For reproducibility, the target motion is physically generated using another robot manipulator. As shown in Figs. 14 and 15, the robot successfully follows the target trajectory, confirming that the proposed framework can effectively handle dynamic targets. It is also observed that when the target moves at relatively high speed in the 2D workspace, tracking failures may occur

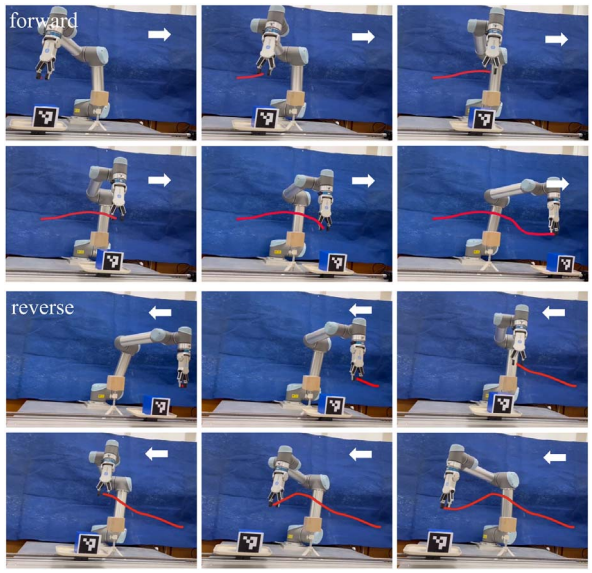


Fig. 12. Frame-by-frame plots under the 1-D dynamic target tracking task using our proposed approach.

TABLE III
COMPARISON OF SRS UNDER SAFE DYNAMIC TARGET TRACKING TASK

Policy	SR (%)	Policy	SR (%)	Policy	SR (%)
SAC_Ours	**78.63**	**PPO_Ours**	**72.36**	**TD3_Ours**	**74.70**
SAC_BF	76.58	PPO_BF	70.98	TD3_BF	71.75
SAC_VB	75.43	PPO_VB	70.54	TD3_VB	74.36
SAC	76.43	PPO	71.43	TD3	45.36

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

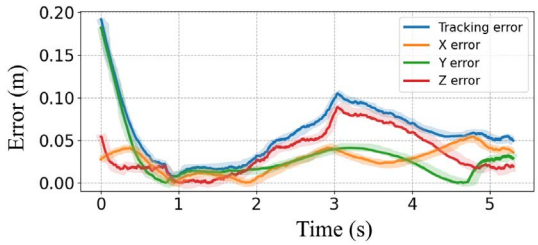


Fig. 13. Real-time tracking errors of the end-effector under the 1-D dynamic target tracking task using our proposed approach.

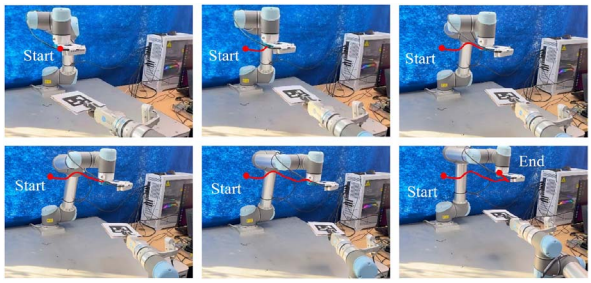


Fig. 14. Frame-by-frame plots under the 2-D dynamic target tracking task using the proposed approach.

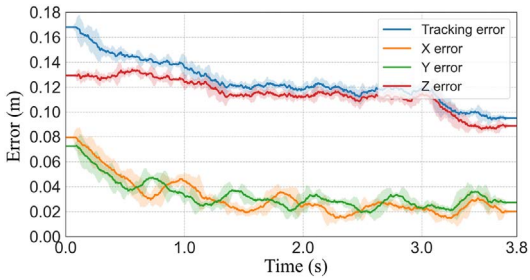


Fig. 15. Real-time tracking errors of the end-effector under the 2-D dynamic target tracking task using our proposed approach.

due to limited computational bandwidth, which highlights an important direction for further optimization in future work.

Furthermore, dynamic target tracking experiments are also conducted using PPO and TD3. Both methods tend to fail frequently in practice, and the few successful cases exhibit performance patterns similar to those observed in the reaching task. Therefore, specific plots for these methods are not included here.

V. Conclusion

This article has presented a model-free framework for safe and smooth visual servoing control of robot manipulators by integrating collision detection and trajectory blending schemes with RL. Unlike prior works, enhanced safety has been achieved by embedding collision avoidance directly into the reward design of the learning process. In addition, a trajectory blending strategy that combines a BF with polynomial interpolation has been developed to mitigate abrupt variations in joint velocities and ensure smooth motion execution. Experimental evaluations have demonstrated that the proposed method enables efficient visual servoing with collision avoidance.

Nevertheless, potential limitations remain in scenarios involving visual occlusion or sim-to-real transfer within complex environments. Future work may address these challenges by incorporating multimodal sensing, predictive mechanisms, and adaptive safety-aware control to further enhance generalization in dynamic and challenging tasks.

References

[1] J. Su, L. Wang, C. Liu, and H. Qiao, "Robotic inserting a moving object using visual-based control with time-delay compensator," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 1842–1852, Feb. 2024.

[2] Y. Wang, Y. Liang, D. Chen, Y. Liu, and M. Wang, "A goods sorting robot system for e-commerce logistics warehouse based on robotic arm technology," in *Proc. IEEE Int. Conf. Real-Time Comput. Robot. (RCAR)*, 2020, pp. 310–314.

[3] P. Zhou, R. Peng, M. Xu, V. Wu, and D. Navarro-Alarcon, "Path planning with automatic seam extraction over point cloud models for robotic arc welding," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5002–5009, Jul. 2021.

[4] K. Li, H. Zhang, and H. Wang, "Nonlinear observer-based visual servoing and vibration control of flexible robotic manipulators with a fixed camera," *IEEE Trans. Cybern.*, vol. 54, no. 7, pp. 4100–4110, Jul. 2024.

[5] Y. Chen et al., "Image-based visual servoing of unmanned aerial manipulators for tracking and grasping a moving target," *IEEE Trans. Ind. Informat.*, vol. 19, no. 8, pp. 8889–8899, Aug. 2023.

[6] P. Serra, R. Cunha, T. Hamel, D. Cabecinhas, and C. Silvestre, "Landing of a quadrotor on a moving target using dynamic image-based visual servo control," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1524–1525, Dec. 2016.

[7] J. Lin, Y. Wang, Z. Miao, S. Fan, and H. Wang, "Robust observer-based visual servo control for quadrotors tracking unknown moving targets," *IEEE/ASME Trans. Mechatronics*, vol. 28, no. 3, pp. 1268–1279, Jun. 2023.

[8] J. Yang, X. Liu, J. Sun, and S. Li, "Sampled-data robust visual servoing control for moving target tracking of an inertially stabilized platform with a measurement delay," *Automatica*, vol. 137, 2022, Art. No. 110105.

[9] H. Shi, G. Sun, Y. Wang, and K.-S. Hwang, "Adaptive image-based visual servoing with temporary loss of the visual signal," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 1956–1965, Aug. 2018.

[10] I. Siradjuddin, L. Behera, T. M. McGinnity, and S. Coleman, "Image-based visual servoing of a 7-DoF robot manipulator using an adaptive distributed fuzzy pd controller," *IEEE/ASME Trans. Mechatronics*, vol. 19, no. 2, pp. 512–523, Mar. 2013.

[11] H. M. Becerra, G. López-Nicolás, and C. Sagüés, "A sliding-mode-control law for mobile robots based on epipolar visual servoing from three views," *IEEE Trans. Robot.*, vol. 27, no. 1, pp. 175–183, Feb. 2011.

[12] A. Hajiloo, M. Keshmiri, W.-F. Xie, and T.-T. Wang, "Robust online model predictive control for a constrained image-based visual servoing," *IEEE Trans. Ind. Electron.*, vol. 63, no. 4, pp. 2242–2250, Dec. 2015.

[13] T. Zhu, J. Mao, L. Han, C. Zhang, and J. Yang, "Real-time dynamic obstacle avoidance for robot manipulators based on cascaded nonlinear MPC with artificial potential field," *IEEE Trans. Ind. Electron.*, vol. 71, no. 7, pp. 7424–7434, Aug. 2023.

[14] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, pp. 737–744, 2020.

[15] H. Liu, F. Ying, R. Jiang, Y. Shan, and B. Shen, "Obstacle-avoidable robotic motion planning framework based on deep reinforcement learning," *IEEE/ASME Trans. Mechatronics*, vol. 29, no. 6, pp. 4377–4388, Dec. 2024.

[16] D. Kalashnikov et al., "Scalable deep reinforcement learning for vision-based robotic manipulation," in *Proc. 2nd Int. Conf. Robot Learn.*, 2018, pp. 651–673.

[17] T. Zhang, K. Zhang, J. Lin, W.-Y. G. Louie, and H. Huang, "Sim2real learning of obstacle avoidance for robotic manipulators in uncertain environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 1, pp. 65–72, Sep. 2021.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.

[18] G. Schoettler et al., "Deep reinforcement learning for industrial insertion tasks with visual inputs and natural rewards," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 5548–5555.
[19] S. Luo, H. Kasaei, and L. Schomaker, "Self-imitation learning by planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 4823–4829.
[20] A. Saxena, H. Pandya, G. Kumar, A. Gaud, and K. M. Krishna, "Exploring convolutional networks for end-to-end visual servoing," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 3817–3823.
[21] J. Guo, H.-T. Nguyen, C. Liu, and C. C. Cheah, "Convolutional neural network-based robot control for an eye-in-hand camera," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 53, no. 8, pp. 4764–4775, Aug. 2023.
[22] N. Yogeswaran et al., "New materials and advances in making electronic skin for interactive robots," *Adv. Robot.*, vol. 29, no. 21, pp. 1359–1373, 2015.
[23] L. Han, J. Mao, P. Cao, Y. Gan, and S. Li, "Toward sensorless interaction force estimation for industrial robots using high-order finite-time observers," *IEEE Trans. Ind. Electron.*, vol. 69, no. 7, pp. 7275–7284, Jul. 2021.
[24] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2012, pp. 3859–3866.
[25] A. Coulombe and H.-C. Lin, "High precision real time collision detection," 2021, *arXiv:200712045v2*.
[26] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
[27] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement learning with deep energy-based policies," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1352–1361.
[28] C. Lv, W. Lin, and B. Zhao, "Voxel structure-based mesh reconstruction from a 3D point cloud," *IEEE Trans. Multimedia*, vol. 24, pp. 1815–1829, Apr. 2021.
[29] F. Giuliari, I. Hasan, M. Cristani, and F. Galasso, "Transformer networks for trajectory forecasting," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, IEEE, 2021, pp. 10335–10342.
[30] K. Katyal, K. Popek, C. Paxton, P. Burlina, and G. D. Hager, "Uncertainty-aware occupancy map prediction using generative networks for robot navigation," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, IEEE, 2019, pp. 5453–5459.
[31] Y. Zhao, J. Mao, and C. Zhang, "Safe reinforcement learning-based dynamic obstacle avoidance for robotic manipulators," in *Proc. 10th Int. Conf. Automat., Control Robot. Eng. (CACRE)*, IEEE, 2025, pp. 5453–5459.
[32] J. W. Mock and S. S. Muknahallipatna, "A comparison of PPO, TD3 and SAC reinforcement algorithms for quadruped walking gait generation," *J. Intell. Learn. Syst. Appl.*, vol. 15, no. 1, pp. 36–56, 2023.



Jianliang Mao (Member, IEEE) received the B.Sc. degree in automation, the M.Sc. degree in control engineering, and the Ph.D. degree in control theory and control engineering from the School of Automation, Southeast University, Nanjing, China, in 2011, 2014, and 2018, respectively.

From 2018 to 2021, he worked with the Research and Development Institute, Estun Automation Company, Ltd, Nanjing, China. Since 2021, he has been with the College of Automation Engineering, Shanghai University of Electric Power, Shanghai, China. His research interests include model predictive control, sliding mode control, vision-based interactive control, and their applications to electric drives and robot manipulators.



Jun Hu received the B.Sc. degree in mechanical engineering from Taihu University of Wuxi, Wuxi, China, in 2022, and is currently working toward the M.E. degree in computer technology with Shanghai University of Electric Power, Shanghai, China.

His research interests include motion planning, reinforcement learning, and visual servoing.



Xin Zhou (Member, IEEE) received the B.E. degree in electrical engineering and automation from Xi'an Jiaotong-Liverpool University, Suzhou, China, in 2015, the M.S. degree in digital systems and communication engineering from Australian National University, Canberra, Australia, in 2017, and the Ph.D. degree in control science and engineering from East China University of Science and Technology, Shanghai, China, in 2022.

Since August 2022, she has been with the College of Automation Engineering, Shanghai University of Electric Power, Shanghai, China. Her research interests include multiobjective intelligent optimization, path planning, trajectory planning, and their applications to high-dimensional manipulators.



Chuanlin Zhang (Senior Member, IEEE) received the B.S. degree in mathematics and the Ph.D. degree in control theory and control engineering from the School of Automation, Southeast University, Nanjing, China, in 2008 and 2014, respectively. From 2011 to 2012, he was a Visiting Ph.D. Student with the Department of Electrical and Computer Engineering, University of Texas, San Antonio, TX, USA. From 2016 to 2017, he was a Visiting Scholar with the Energy Research Institute, Nanyang Technological University, Singapore.

Since 2014, he has been with the College of Automation Engineering, Shanghai University of Electric Power, Shanghai, China, where he is currently a Professor.

His research interests include nonlinear system control theory and applications for power systems.



Jun Yang (Fellow, IEEE) received the B.Sc. degree in automation from the Department of Automatic Control, Northeastern University, Shenyang, China, in 2006, and the Ph.D. degree in control theory and control engineering from the School of Automation, Southeast University, Nanjing, China, in 2011.

Since 2020, he has been with the Department of Aeronautical and Automotive Engineering, Loughborough University, Loughborough, U.K., as a Senior Lecturer. His research interests include disturbance observer, motion control, visual servoing, nonlinear control, and autonomous systems.

Dr. Yang was the recipient of the EPSRC New Investigator Award. He serves as an Associate Editor or Technical Editor for IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, IEEE/ASME TRANSACTIONS ON MECHATRONICS, and *IEEE Open Journal of the Industrial Electronics Society*.



Hesheng Wang (Senior Member, IEEE) received the B.E. degree in electrical engineering from Harbin Institute of Technology, Harbin, China, in 2002, and the M.Phil. and Ph.D. degrees in automation and computer-aided engineering from The Chinese University of Hong Kong, Hong Kong, China, in 2004 and 2007, respectively.

He is currently a Distinguished Professor with the School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Shanghai, China. His research interests include visual servoing, service robots, computer vision, and autonomous driving.

Dr. Wang is an Associate Editor of *Robotic Intelligence and Automation*, and the *International Journal of Humanoid Robotics*, and a Senior Editor for the IEEE/ASME TRANSACTIONS ON MECHATRONICS, an Editor-in-chief of Robot Learning. From 2015 to 2019, was an Associate Editor for IEEE TRANSACTIONS ON ROBOTICS and IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING, from 2021 to 2023. He was the General Chair of IEEE/RSJ IROS 2025, IEEE ROBIO 2022, and IEEE RCAR 2016.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 30,2026 at 07:29:47 UTC from IEEE Xplore. Restrictions apply.